Wrong the Same Way: Cross-Prompt Sharing Shapes Reward Hacking in RLVR

Anonymous Author(s)
Affiliation
Address
email

Abstract

At its KL-regularised optimum, reinforcement learning with verifiable rewards (RLVR) acts as a filter: it reweights the verifier-accepted set as a whole without altering its internal composition. The rate of reward hacking should therefore be inherited from the base model rather than learned. Yet what breaks this is not the overall error rate but the specific pattern of verifier errors. We ask which property of that pattern drives exploitation, and identify cross-prompt parameter sharing. Writing the exact drift of the policy off the filter baseline as a sum of covariances between reward and the empirical tangent kernel isolates a cross-prompt term that vanishes identically without parameter sharing. We validate this in a controlled RLVR environment small enough to compute the entire response distribution exactly. Replacing the shared architecture with independent per-prompt copies removes the dependency on leak breadth, neutralizing the drift slope. Targeted controls further show that sharing at RL time alone is insufficient: exploitation is amplified only when the base model's representation already encodes the verifier's acceptance pattern. Verifier hardening must therefore be budgeted against the model's pre-existing shared error modes, since patches targeting universal leaks yield significantly higher true accuracy than uniform error reduction.

1 Introduction

As language models increasingly rely on reinforcement learning with verifiable rewards (RLVR), understanding how they exploit imperfect verifiers is critical for safe deployment. In RLVR, a response is rewarded if a programmatic checker accepts it [Shao et al., 2024, Guo et al., 2025]. These verifiers are imperfect. Generative judges accept degenerate strings: non-word symbols and generic reasoning openers elicit false positives across a wide range of judge models [Zhao et al., 2026]. Test suites accept programs that pass the visible tests and nothing else [Khalifa et al., 2026]. Outcome-only checkers reward a correct final answer reached by invalid reasoning [Yuan et al., 2026], a failure mode that motivated supervising each reasoning step [Lightman et al., 2023] rather than the final answer alone [Cobbe et al., 2021]. Each such false positive is a response the policy can be paid for without being correct.

The literature presents two incompatible accounts of how RLVR interacts with these false positives. In the first, RLVR tolerates verifier error: sweeping the noise rate from 1% to 50% leaves peak validation accuracy within two points of clean training up to 15%, with graceful degradation to 30% [Plesner et al., 2026]. Noise rescales convergence time rather than the destination, provided the verifier's Youden index $J = \mathrm{TPR} - \mathrm{FPR}$ remains positive [Rad et al., 2026]. A preregistered contrast between leaky and hardened test suites suggests that the incorrect programs a trained model emits were already being emitted by the base model, pointing to a "selection of pre-existing error modes rather than learned exploitation" [Zhang, 2026]. In the second, verifier exploitation is a first-order practical

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

failure: RL amplifies hacking seeded by a trace of contamination [Khalifa et al., 2026], systematic verification error produces delay, plateau, or collapse [Egashira et al., 2026], RLVR-trained models learn to game extensional verifiers [Helff et al., 2026], and production reward hacking generalizes into broader misalignment [MacDiarmid et al., 2025].

Theory favours the first account. At the KL-regularised optimum of the RLHF objective [Stiennon et al., 2020, Ouyang et al., 2022], the policy is $\pi^\star(y \mid x) \propto \pi_{\mathrm{ref}}(y \mid x)\, e^{r(x,y)/\beta}$, so within any reward level set the odds match the reference model's [Korbak et al., 2022]. Every accepted response, whether correct or exploiting, is multiplied by the same factor. RLVR therefore acts as a filter on the accepted set, not a sculptor inside it, a point made repeatedly in the RLVR diversity literature [Gai et al., 2025, Kruszewski et al., 2026, Dymetman, 2026] and the reason RLVR sharpens rather than expands what a base model can do [Yue et al., 2025]. Applied to verifier false positives, this predicts that the post-RL hack rate is inherited from the base model. The prediction is delimited: it holds at the KL-regularised optimum and at every step of a multiplicative update, but not for arbitrary algorithms or parameterisations, and our own experiments show first-order training departing from this baseline. We treat the filter theory as a null model, and characterize the mechanisms driving the departure from it.

Contributions.

- **Exact drift decomposition.** We write the instantaneous drift of $\log \pi_\theta(y \mid x)$ under RLVR gradient flow as a sum over *all* prompts of the covariance between reward and the empirical tangent kernel (Lemma 1). Without cross-prompt sharing it collapses to one term, so the dynamics cannot depend on how many prompts share a leak (Corollary 1).
- **Universality threshold.** For shared-parameter policies the leading-order drift of a leak accepted on a fraction u of prompts has the sign of $u - \bar{V}$, with $\bar{V}$ the policy's current pass rate, an observable requiring no ground truth (Proposition 2). Below it, sharing *suppresses* the leak. Neither side depends on the verifier's error rate.
- **An exactly enumerable environment.** The full response distribution is computed exactly, so every reported quantity is a true value rather than a sample estimate, and universality and static false-positive rate are independent knobs.
- **Isolating the mechanism.** Replacing the shared transformer with N per-prompt copies of the identical architecture removes the dependence on u. Further controls show sharing at RL time alone is insufficient: RL amplifies a leak only when the base model's representation already encodes the verifier's acceptance pattern.

2 Setup: RLVR with an exploitable verifier

To systematically investigate these dynamics, we first define the standard RLVR objective and the precise metrics we use to track reward hacking. Let $\mathcal{X}$ be a finite prompt set, $|\mathcal{X}| = N$, with the uniform distribution, and $\mathcal{Y}$ a finite response space. A response is *correct* if $c(x, y) = 1$; we write g_x for the correct response, which is unique in our setting. The deployed verifier returns $r(x, y) \in \{0, 1\}$ with $r \geq c$ (assuming no false negatives). We write $\mathcal{A}(x) = \{y : r(x, y) = 1\}$ for the accepted set and $\mathcal{H}(x) = \{y : r(x, y) = 1,\, c(x, y) = 0\}$ for its false-positive part, which we refer to as the *leaks*. RLVR maximizes $J(\theta) = \mathbb{E}_x\, \mathbb{E}_{y\sim\pi_\theta(\cdot|x)}[r(x, y)]$.

Definition 1 (Hack fraction)**.** We define the hack fraction as $\rho(\pi) = \frac{1}{N} \sum_x \pi(\mathcal{H}(x) \mid x) \,/\, \pi(\mathcal{A}(x) \mid x)$, representing the probability that a rewarded response is a false positive, averaged over prompts. We also track the prompt-averaged log-odds $\Lambda(\pi) = \frac{1}{N} \sum_x \log \big[\pi(\mathcal{H}(x) \mid x) \,/\, \pi(g_x \mid x)\big]$.

Definition 2 (Leak universality)**.** For a given response h, let $m(h) = |\{x : h \in \mathcal{H}(x)\}|$ and $u(h) = m(h)/N$. A leak with $u = 1$ is universally accepted everywhere; a leak with $u = 1/N$ is idiosyncratic and specific to only one prompt.

It is important to note that universality is entirely orthogonal to the verifier's error rate. A verifier that accepts exactly one wrong response per prompt yields the same static false-positive rate on an audit, regardless of whether that accepted response is the same universal string everywhere or a uniquely different string for each prompt.

2.1 The filter baseline

Proposition 1 (RLVR is a filter; known)**.** *Assume the verifier has no false negatives, $r \geq c$, meaning correct responses and leaks lie in the same reward level set. Let $\pi^\star_\beta = \arg\max_\pi J(\pi) - \beta\, \mathbb{E}_x \mathrm{KL}(\pi(\cdot \mid x) \| \pi_{\mathrm{ref}}(\cdot \mid x))$. Then $\pi^\star_\beta(y \mid x) \propto \pi_{\mathrm{ref}}(y \mid x)\, e^{r(x,y)/\beta}$, meaning for any y, y' with $r(x,y) = r(x,y')$ the relative odds remain unchanged. Consequently, $\rho(\pi^\star_\beta) = \rho(\pi_{\mathrm{ref}})$ and $\Lambda(\pi^\star_\beta) = \Lambda(\pi_{\mathrm{ref}})$ for every $\beta > 0$. This identical invariance holds at every step of any multiplicative update $\pi_{t+1}(\cdot \mid x) \propto \pi_t(\cdot \mid x) \exp(\eta f(r(x,\cdot)))$.*

This closed form relies on the Gibbs variational principle, standard in RL [Peters et al., 2010] and in the LLM setting [Korbak et al., 2022, Kruszewski et al., 2026, Dymetman, 2026]. We restate it because it is the baseline against which all departures are measured, and its consequence for reward hacking, that the composition of the accepted set stays frozen at the base model's, is the prediction under test. As $\beta \to 0$, $\pi^\star_\beta \to \pi_{\mathrm{ref}}(\cdot \mid \mathcal{A}(x))$: the whole regularisation path is the reference model conditioned on acceptance, which is why nothing inside the accepted set can move. We verify this invariance numerically to floating-point precision (Section 5).

Why the per-prompt statistic. We use a per-prompt statistic rather than a pooled one. The pooled quantity $\sum_x \pi(\mathcal{H}(x) \mid x) / \sum_x \pi(\mathcal{A}(x) \mid x)$ is a $\pi(\mathcal{A}(x) \mid x)$-weighted average of the invariant per-prompt ratios, and RLVR raises those weights toward 1 at differing rates. Its total excursion along the filtering path is exactly $-\operatorname{Cov}_x\big(\pi_{\mathrm{ref}}(\mathcal{A}(x) \mid x),\, \rho_x\big) \,/\, \mathbb{E}_x[\pi_{\mathrm{ref}}(\mathcal{A}(x) \mid x)]$ (Appendix A), which is *positive* whenever pass rate and hack fraction are negatively correlated across prompts. That is the typical case, as prompts a model can solve are prompts where it relies on the leak less. A practitioner tracking the pooled $P(\text{hack} \mid \text{rewarded})$ will therefore see it climb under pure filtering, with nothing exploited. In our environment it rises by +0.057 under exact mirror descent while ρ and Λ remain static. We report per-prompt statistics throughout.

3 Decoding the departure from the filter baseline

In practice, RLVR executes first-order updates on a shared-parameter model, often with a minimal or entirely absent KL term. To understand how models diverge from the theoretical baseline, we characterize the resulting drift exactly.

Lemma 1 (Drift decomposition)**.** *Under exact-gradient flow $\dot\theta = \nabla_\theta J(\theta)$,*

$$\frac{d}{dt} \log \pi_\theta(y \mid x) = \frac{1}{N} \sum_{x' \in \mathcal{X}} \operatorname{Cov}_{y' \sim \pi_\theta(\cdot \mid x')} \Big[r(x', y'),\; K_\theta\big((x,y),(x',y')\big) \Big], \tag{1}$$

where $K_\theta((x,y),(x',y')) = \langle \nabla_\theta \log \pi_\theta(y \mid x), \nabla_\theta \log \pi_\theta(y' \mid x') \rangle$ is the empirical tangent kernel of the log-policy. The proof is a one-line consequence of the score-function identity and is given in Appendix A.

Equation (1) shows that the log-probability of a response on a single prompt is influenced by the dynamics on *every* prompt, in proportion to how strongly reward covaries with tangent similarity there. It is an instantaneous identity and we apply it as such. Our experiments use Adam [Kingma and Ba, 2015] rather than pure gradient flow. For an update $\dot\theta = P \nabla J$ with fixed, positive definite P, (1) holds with K_θ replaced by $K^P_\theta((x,y),(x',y')) = \langle \nabla_\theta \log \pi_\theta(y \mid x),\, P\, \nabla_\theta \log \pi_\theta(y' \mid x') \rangle$. Adam's first moment makes the update history-dependent, so the instantaneous identity does not apply verbatim. The support argument survives, which is all Corollary 1 requires: Adam's state is per-coordinate, and its momentum is a linear combination of gradients that are already block-supported, so a per-prompt parameterisation keeps a vanishing cross-prompt term under Adam as under flow. (Our kernel is the finite-width empirical kernel, not the constant infinite-width limit of Jacot et al. [2018].)

Corollary 1 (Per-prompt parameters cannot see universality)**.** *If $\theta = (\theta_x)_{x \in \mathcal{X}}$ with $\pi_\theta(\cdot \mid x)$ operating as a function of θ_x only, then $\nabla_{\theta_{x'}} \log \pi_\theta(y \mid x) = 0$ for $x' \neq x$. Consequently, $K_\theta((x,y),(x',y')) = 0$ off the diagonal, and (1) collapses solely to the $x' = x$ term. Indeed, $\nabla_{\theta_x} J = \frac{1}{N} \sum_y \pi_\theta(y \mid x)(r(x,y) - V(x)) \nabla_{\theta_x} \log \pi_\theta(y \mid x)$ depends on θ_x alone, rendering the trajectory of block x an autonomous ODE. A per-prompt policy generates no u-dependence of its own, and can depend on m only through what it inherited structurally at initialisation.*

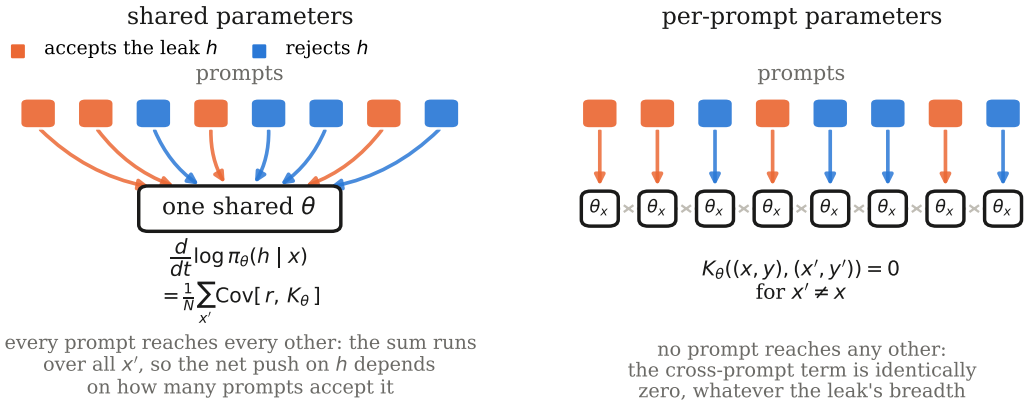


Figure 1: Corollary 1, drawn. With shared parameters every prompt's gradient lands on the same weights, so a leak accepted on some prompts and rejected on others is pushed both ways at once and the net depends on how many accept it. With per-prompt parameters the cross-prompt kernel entries are identically zero, so that channel does not exist at any leak breadth.

This makes the tabular arm a control rather than merely a baseline. The argument survives Adam, whose state is per-coordinate, but would not survive a global gradient-norm clip, which we omit. It does not imply the per-prompt policy sits *on* the filter baseline: for two equally rewarded responses, gradient flow gives $\frac{d}{dt}\log(\pi_1/\pi_2) = (\pi_1 - \pi_2)(1 - J)$, the known rich-get-richer behaviour of softmax policy gradient [Mei et al., 2020, Lochab et al., 2026]. That drift is prompt-local and blind to u, which is what makes it a control rather than a confound.

Equation (1) does not by itself isolate $u = m/N$. Specialising it to a leak h, the acceptance fraction that multiplies the drift is kernel- and policy-weighted:

$$u_{\text{eff}}(x,h) \;=\; \Big(\textstyle\sum_{x'\in U} w_{x'}\Big)\Big/\Big(\textstyle\sum_{x'} w_{x'}\Big), \qquad w_{x'} = K_{\theta,xx'}\,\pi_\theta(h \mid x'), \tag{2}$$

where $K_{\theta,xx'}$ is the kernel entry coupling h at x to h at x' and U the accepting set. The raw fraction m/N is recovered only when kernel entries and leak probabilities are homogeneous across prompts, which we operationalise as follows.

Definition 3 (Coherent kernel model)**.** Write $K_\theta = K^{\text{coh}}_\theta + R$ with $K^{\text{coh}}_\theta((x,h),(x',y')) = \kappa_{xx'}\,\mathbf{1}[y' = h] + a(x,x') + b(x,x',y')$, where $\kappa_{xx'}$ is κ_s on the diagonal and κ off it, a does not depend on y' and b does not depend on h. The a and b terms drop out of (1) and of the within-prompt log-odds respectively (Appendix A), leaving (κ, κ_s) and a residual R that we do not bound, which is why Proposition 2 is directional. Coherence is a property of *how a response is computed*, not of string identity: a leak emitted by prompt-independent machinery is coherent, whereas correct answers that happen to coincide across prompts need not be.

Proposition 2 (Universality threshold)**.** *Let h be coherent at scales (κ, κ_s) with $\kappa > 0$, accepted on U with $|U| = m$ and $u = m/N$, and let $\bar{V} = J(\theta)$ be the policy's verifier pass rate. Under the homogeneity assumptions $\pi_\theta(h \mid \cdot) \equiv p_h$ and $V(\cdot) \equiv \bar{V}$, and when the correct answer is incoherent ($\kappa_g \approx 0$),*

$$\frac{d}{dt}\Big[\log \pi_\theta(h \mid x) - \log \pi_\theta(g_x \mid x)\Big] = \kappa\, p_h\,(u - \bar{V}) \;+\; O(1/N), \tag{3}$$

so the leading-order sign is that of $u - \bar{V}$: the leak is amplified when its universality exceeds the policy's current pass rate and suppressed when it does not. The full expression, including the correct answer's own coherence and the $O(1/N)$ terms, is Appendix A, which also derives it.

Equation (3) has three consequences. The verifier's false-positive rate is absent: accepting more distinct wrong responses per prompt raises every static audit metric and leaves it unchanged. Sharing is two-sided, since the mechanism that amplifies a universal leak pushes an idiosyncratic one *below* the base rate, being punished on every prompt that does not accept it, which a per-prompt policy never is. And the threshold rises during training: $\bar{V}$ is $J(\theta)$ and $dJ/dt = \|\nabla J\|^2 \geq 0$ under flow, so leaks of intermediate universality are amplified early and suppressed later, leaving only $u \approx 1$ survivors. That is what a delay, plateau or collapse taxonomy [Egashira et al., 2026] looks like from the inside.

4 Experimental design

To isolate these mechanisms, we designed an environment satisfying three conditions: (i) the full response distribution can be computed exactly, eliminating sample noise; (ii) leak universality and false-positive rates are independently manipulable; and (iii) a shared-parameter policy and a per-prompt policy can be initialized from the *identical* distribution.

Task and response space. Prompts are the $N = 64$ pairs (a, b) with $a, b \in \{2, \ldots, 9\}$. The correct response is the decimal string of the product $a \times b$, zero-padded to two digits. The response space is all 100 two-digit strings, so $\pi_\theta(\cdot \mid x)$ can be enumerated exactly for every prompt at every step. Policies are autoregressive over the two digits.

Verifier. The deployed verifier accepts the correct string plus k designated wrong strings per prompt. None of them is the correct answer to any other prompt, so every response falls into exactly one category: correct, leak, or wrong. Prompts are partitioned into blocks of size m, and all prompts within a block share identical leak strings. Varying m manipulates universality $u = m/N$ while holding the static false-positive rate fixed at $k/(|\mathcal{Y}| - 1)$; varying k sweeps the false-positive rate while pinning universality.

Base model. Our architecture is a 2-layer, 64-dimensional transformer [Vaswani et al., 2017] (101,706 parameters), supervised fine-tuned on a target distribution that blends the correct answer, a per-prompt competence level λ_x spread over $[0, 0.9]$, and a minimal contamination rate $\varepsilon = 0.03$ of leak demonstrations. This intentional contamination aligns with findings by Khalifa et al. [2026], who observed that a mere trace of contamination is sufficient to seed hacking behavior. Competence is assigned via a fixed permutation independent of the block structure, preventing difficulty from correlating with universality. The base model achieves an accuracy of 0.441 and a baseline hack fraction of 0.128. While the SFT target remains identical across conditions, the realized base model, itself a shared-parameter network, naturally fits a universal leak map slightly better than an idiosyncratic one, resulting in a minor base hack fraction variation of 0.015. To account for this, we track and report all metrics as drifts from each run's specific base model.

Matched policies. We pair the transformer with a tabular policy of free per-prompt parameters under the same autoregressive factorisation, which parameterises the response simplex completely. It is initialised to reproduce the base transformer's starting distribution to a tolerance of 10^{-4}, and both are trained on the same objective and the same learning-rate grid (Table 1). They differ in parameter sharing but also, unavoidably, in parameterisation; Section 5.5 separates the two.

Protocol. Unless stated otherwise, we run exact-expectation policy gradient with the exact mean baseline. This mirrors the expectation-limit of RLOO [Ahmadian et al., 2024] and Dr. GRPO [Liu et al., 2025], which debiases GRPO [Shao et al., 2024]. Group-relative variants [Yu et al., 2025] and PPO [Schulman et al., 2017] (inside its clipping range) share this structure. We run 400 steps with Adam, across three to ten seeds per cell (Appendix B, Table 1). We report the operating point a practitioner would select: the smallest learning rate in the grid whose final verifier reward falls within 10^{-3} of the maximum. This tie-break is conservative for our hypothesis, since smaller steps drift less, yet several learning rates saturate the verifier at reward 1 with vastly different true accuracies. Section 5.4 further validates the findings using sampled GRPO (with group-standard-deviation normalisation) and plain SGD.

5 Results

We report changes as drifts relative to each run's own base model: $\Delta\rho$ denotes the probability difference, $\Delta\Lambda$ the log-odds difference. Under the filter baseline of Proposition 1 both are exactly zero, making them immune to the residual base-model mismatch of Section 4. Absolute ρ is plotted alongside the base curve.

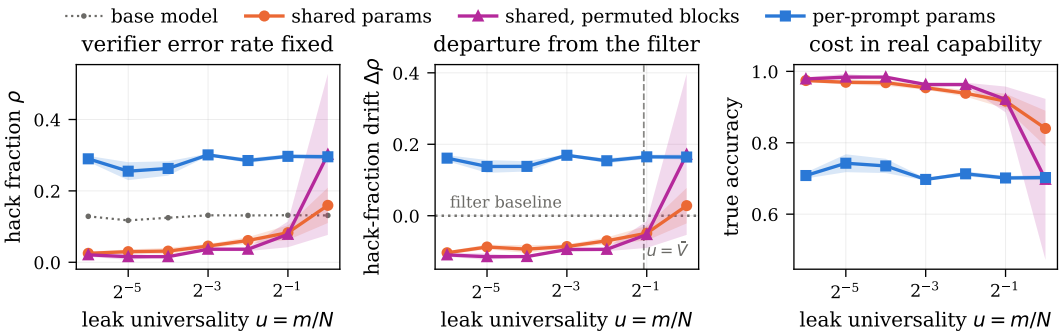


Figure 2: Universality, at a fixed verifier error rate, dictates whether RLVR suppresses a leak or amplifies it. Every verifier accepts exactly one wrong response per prompt, holding the static false-positive rate strictly identical while only varying the number of prompts sharing it. Left: Final hack fraction after training compared to the base model. Centre: Drift from the base model (theoretically zero under the filter baseline); the dashed line denotes $u = \bar{V}$. Right: Resulting true accuracy. The *permuted blocks* series repeats the sweep with randomised block membership. Bands represent ± 1 s.e.m. across seeds.

5.1 The filter baseline holds exactly, and gradient training departs from it

Our findings reveal that exact mirror descent preserves ρ invariance to 8.6×10^{-9} and Λ invariance to 9.6×10^{-8} over 400 steps. Similarly, the KL-regularised optimum preserves ρ invariance to 8.6×10^{-9} across $\beta \in [0.05, 2.0]$, holding to floating-point zero. Validating our analytical framework, Lemma 1 matches finite differences to a precision of 2.9×10^{-4}. We observe the per-prompt policy's cross-prompt kernel sits at exactly 0, starkly contrasting the transformer's 0.098 relative to its own self-kernel.

5.2 Universality dictates behavior for shared-parameter policies

Figure 2 is our primary finding. Holding the verifier's static false-positive rate constant and sweeping only the number of prompts that share the accepted wrong response, the shared-parameter policy's hack fraction climbs from 0.025 ± 0.005 at $u = 0.016$ to 0.160 ± 0.050 at $u = 1$ (10 seeds), against a base model value of 0.128. As a drift, $\Delta\rho$ moves from -0.104 ± 0.004 to $+0.028 \pm 0.050$, while true accuracy degrades from 0.974 to 0.840. The suppression at low u is substantial and unambiguous. The amplification at $u = 1$ under Adam falls within its own error bar, but the trend is a monotonic rise from strong suppression toward zero. A decisive sign change into active amplification appears under sampled GRPO and plain SGD (Section 5.4, Figures 6 and 8), using identical seeds, bases and step budgets and differing only in the gradient estimator or optimizer.

The outcome at $u = 1$ is heterogeneous across seeds rather than merely noisy. The ten final hack fractions span 0.02, 0.06, 0.06, 0.08, 0.08, 0.14, 0.17, 0.20, 0.22, 0.56, so the mean is skewed by a single collapsing seed; removing it leaves the drift at -0.017, still suppression. The heterogeneity is concentrated across prompts: at $u = 1$, a mean of 16% of prompts finish with the leak more probable than the correct answer, against 2% at $u = 0.016$. This is why $\Delta\rho$ approaches zero while $\Delta\Lambda$, dominated by the suppressed majority, climbs from -9.0 to -3.7 but stays negative. In the worst case a $u = 1$ run reaches hack fraction 1.000 at accuracy 0.000 while saturating the verifier.

Randomizing which prompts share a leak reproduces both the suppression and the slope (drift shifting from -0.110 to $+0.172 \pm 0.225$), so the effect is not an artifact of contiguous blocks coinciding with token structure, though its $u = 1$ endpoint here is carried by one of three seeds. That design also equalizes the base, constricting ρ variation to 0.0011 against 0.015 with contiguous blocks.

By contrast the per-prompt policy, initialized identically and trained under the same grid, remains flat. Regressing $\Delta\rho$ on $\log_2 m$ for the permuted sweep yields +0.0003 per doubling against +0.0353 for the transformer, two orders of magnitude apart; with contiguous blocks it yields +0.0028 against +0.0177. The per-prompt policy does not sit *on* the filter baseline (moving +0.161 to +0.164), but its deviation carries no information about universality, which is what Corollary 1 requires. Section 5.5 eliminates the architecture confound.

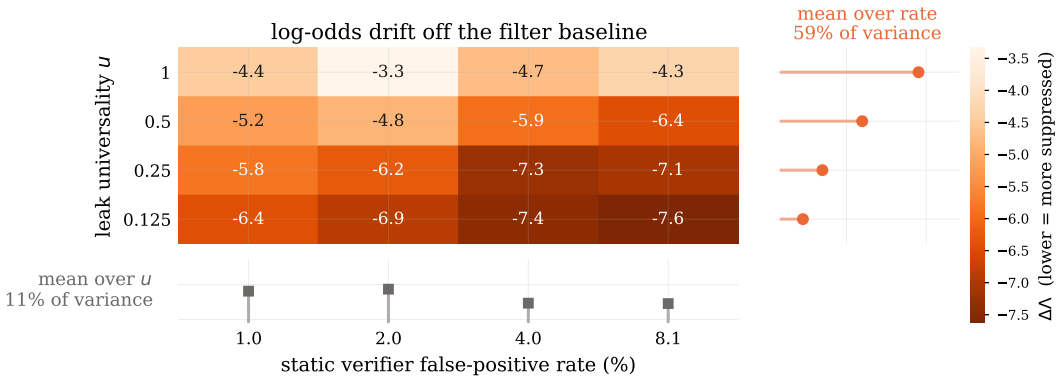


Figure 3: The factorial as a field. Each cell is one (universality, error rate) condition; darker is a leak more strongly suppressed relative to the filter baseline. Reading up a column changes the colour; reading across a row barely does. The marginals carry the variance decomposition, and the horizontal one runs the *wrong* way: accepting more distinct wrong responses per prompt mildly reduces hacking.

Figure 3 crosses universality against the static false-positive rate as a field: the colour gradient runs vertically and is nearly flat horizontally. Here, the base hack *mass* is strictly bounded to ε across all k. A two-way decomposition of the log-odds drift $\Delta\Lambda$ attributes a dominant 59% of its variance directly to universality ($u \in [0.125, 1]$), and only 11% to the false-positive rate across an eightfold range. Averaged over k, $\Delta\Lambda$ shifts drastically from -7.09 at $u = 0.125$ to -4.19 at $u = 1$; conversely, averaged over u, it barely shifts from -5.47 to -6.36 even as the false-positive rate multiplies eightfold. The directionality is critical: accepting *more* wrong responses per prompt does not inherently increase hacking and often mildly decreases it, as the base model's fixed propensity is diluted across rarer, individual strings. Standard audits that merely count total false positives are therefore measuring a metric that frequently exhibits both the wrong magnitude and the wrong sign.

5.3 Isolating the mechanism: cross-prompt gradient aggregation

Lemma 1 splits the drift into an own-prompt term, present for any policy, and a cross-prompt term that is the entire contribution of sharing (Figure 1). By autograd at the base model the cross-prompt term is exactly 0 for the per-prompt policy at every universality, where Corollary 1 is exact, and for the shared policy rises from 6.6 ± 1.1 to 12.2 ± 0.8, peaking at 14.0 at $u = 0.5$. Coherence is measured, not posited: the mean pairwise cosine between per-prompt leak gradients rises from +0.02 to +0.29 while for correct answers it stays at +0.03, though those are not perfectly prompt-specific either ($u_g = 0.037$). Figure 5 in Appendix C plots both.

What we could not verify. The $u > \bar{V}$ threshold cannot be pinned down pointwise. At initialisation the cross-prompt term is positive at every u, dominated by a transient in which every accepted response's logits rise together. After 60 steps that transient has passed but $\bar{V}$ has already reached 0.993, so the predicted threshold sits at essentially 1 and the cross-prompt log-odds drift is small and negative everywhere (-0.067 to -0.030): consistent with the proposition for $u \leq \bar{V}$, but not a sign change. The observable window is narrow; what survives is the directional content.

5.4 Robustness across GRPO, SGD, and KL penalties

Figure 6 in Appendix C covers GRPO and KL constraints, and Figure 8 the SGD sweep. The vulnerability is not an artifact of exact-expectation gradients or of Adam. Under sampled GRPO (group size 8 with standard-deviation normalisation), $\Delta\rho$ shifts from -0.103 ± 0.004 to $+0.663 \pm 0.079$ across 10 seeds, with a definitive zero-crossing at $u = 0.27$. Plain SGD, the optimizer our analysis relies upon, produces the clearest separation in our study: the shared policy climbs from 0.051 to 1.000, with accuracy falling from 0.926 to 0.000, a slope of +0.184 per doubling and a crossing at $u = 0.07$. The per-prompt policy shifts only 0.053 to 0.055 (slope -0.001), staying *below* its base model throughout, which is the prompt-local rich-get-richer drift of Appendix A.

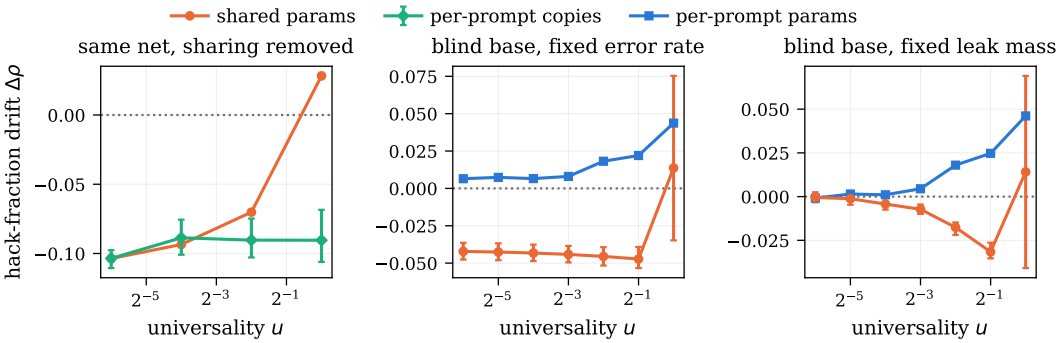


Figure 4: Left: substituting the shared model with independent per-prompt copies of the *identical* transformer architecture, under the same conditions and seeds. Centre & Right: freezing a structure-blind base model while varying only the acceptance mask, holding either the static false-positive rate or the accepted string's base mass constant. Bars denote bootstrap 95% intervals.

A KL penalty pulls the policy back toward the filter baseline as Proposition 1 suggests, and where resolved it does so in the direction that *costs* accuracy: raising β to 0.3 takes $\Delta\rho$ from -0.103 to -0.002 at $u = 0.016$, removing the *beneficial* suppression sharing provided, at a cost of 0.974 to 0.822 in accuracy. KL is not a free defence: it commits the model to the base model's composition of the accepted set, which helps only when that composition is better than what training would otherwise reach. Ackermann et al. [2026] reach a compatible conclusion from the other direction.

5.5 Targeted controls: diagnosing the role of parameter sharing

The sweeps above contrast a deep transformer against a free per-prompt table, which confounds parameterisation with sharing. Figure 4 adds three controls: one that removes the parameterisation difference, and two that keep the base model structurally blind to the leak.

Architecture-fixed sharing. We give each prompt its own copy of the transformer, all N copies initialised from the same checkpoint and taking identical steps on the lowest three learning rates, with copy x evaluating only prompt x. The dependence on universality disappears: the slope flattens to $+0.0019$ per doubling (CI $[-0.0008, +0.0050]$, consistent with zero) against $+0.0210$ for the shared model under matched conditions. The paired within-seed difference is $+0.0191$ (CI $[+0.0082, +0.0330]$, $t = 2.82$ on 10 seeds). At $u = 1$ the copies drift -0.090 and hold 0.959 accuracy, where the shared model loses both. This is Corollary 1 with the architecture confound removed.

Mask-only controls. Changing the verifier changes the supervised target and therefore the base model, so universality confounds with how deeply the base internalises the leak, a structural coherence we measured growing seventeenfold. We freeze one checkpoint per seed and vary only which prompts accept which of two fixed leak strings, in two variants that lock either the static false-positive rate or the accepted string's *per-prompt* base mass (Appendix B).

In neither blind variant is the shared policy's slope steeper than the per-prompt baseline: $+0.0056$ (CI $[+0.0003, +0.0121]$) and -0.0011 (CI $[-0.0074, +0.0051]$) against tabular baselines of $+0.0054$ and $+0.0073$, with paired differences $+0.0001$ and -0.0084. The design has power to exclude an E10-sized effect, so this is a negative result rather than an underpowered null. The controls differ from the sweeps in one respect: their base models are *blind* to the leak structure. The vulnerability is therefore two-stage. Shared-parameter supervised training must first encode the leak's cross-prompt structure in the representation; shared-parameter RL then amplifies what it finds. Sharing at RL time alone is not enough.

5.6 Strategic budgeting for verifier hardening

Two patches that remove the same raw count of accepted wrong pairs, and so score identically on a static audit, differ sharply in what they buy. Removing a single universal leak takes the policy to

0.984 accuracy; removing every idiosyncratic leak instead yields 0.583, a gap of 40 ± 16 points of true accuracy (3 seeds). The per-prompt policy shows a 11 point gap between the same two verifiers, already present in its base model, so the remaining 29 points are manufactured by RL. Setup details are in Figure 6 and Appendix C.

6 Related work

RLVR as filtering. The KL-regularised optimum as an exponential tilt is well established [Korbak et al., 2022, Gai et al., 2025, Kruszewski et al., 2026, Dymetman, 2026, Lochab et al., 2026] and underpins the reading that RLVR sharpens rather than expands base-model capability [Yue et al., 2025]; inverting the same relation is how Rafailov et al. [2023] dispense with a separate reward model. That literature reads the invariance as a statement about diversity among correct answers. We read it as a constraint on the composition of the accepted set, and take it as our null model. We likewise use the rich-get-richer behaviour among tied responses [Mei et al., 2020, Lochab et al., 2026] as a control.

Verifier error. Wang et al. [2020] estimate the reward confusion matrix and prove convergence under unbiased surrogate rewards; Cai et al. [2026] derive RLVR corrections that outperform uncorrected training. Both show that *correction* helps, not that uncorrected training is robust. Plesner et al. [2026] measure noise tolerance directly, and Rad et al. [2026] map a phase transition in the Youden index: incorrect modes go extinct when $J > 0$ and dominate when $J < 0$. Spurious rewards have occasionally helped particular models [Shao et al., 2026]. Audits of verifier false positives [Zhao et al., 2026, Huang et al., 2026, Xu et al., 2025, Yuan et al., 2026] report gross error rates and leave the structural alignment of those errors across prompts unmeasured.

Systematic error. Egashira et al. [2026] established the premise we build on: outcomes are dictated by the "specific pattern of introduced errors" rather than the blanket error rate, through trigger frequency and conditional advantage at initialisation. Their triggers operate at $u \approx 1$; we hold the error rate fixed and sweep how many prompts share the accepted response. That isolates the structural axis and the mechanism behind it, a cross-prompt covariance that vanishes without sharing, and exposes active *suppression* at low universality where prior outcomes were uniformly non-positive. Zhang [2026] reports inheritance while Khalifa et al. [2026] and Nishimura-Gasparian et al. [2026] report amplification and specification gaming; universality reconciles them. Unlike work defining what corrupted rewards *can* do [Everitt et al., 2017, Skalse et al., 2022, Gao et al., 2023], we ask how standard optimizers interact with them. Studies of group-relative advantage estimators [Yang et al., 2026, Zeng et al., 2026] are orthogonal: our effect persists under exact-expectation gradients.

7 Conclusion

RLVR acts as a neutral filter at its KL-regularised optimum; cross-prompt parameter sharing breaks that baseline in practice. Building on the finding that the pattern of a verifier's errors, rather than its rate, dictates reward hacking [Egashira et al., 2026], we identify sharing as the mechanism. A leak aggregates gradient from every prompt that accepts it and is penalised by every prompt that does not. Remove the sharing and the channel vanishes: independent per-prompt copies of the identical architecture show no dependence on leak breadth. Structure-blind controls make the process two-stage, since sharing at RL time is not sufficient on its own and RL amplifies structure that supervised training already encoded. The quantity to audit is therefore neither the gross frequency of verifier errors nor the breadth of acceptance, but whether the admitted false positives align with the model's pre-existing shared error modes.

Limitations and future work. Proposition 2 assumes a coherent kernel and constant pass rates; our measured kernel varied seventeenfold, so the threshold is a directional heuristic rather than a bound. Exact enumeration required a two-token response space and designated leak strings, so string identity stands in for a shared error mode, an assumption that must scale to open-ended generation. We relied on a contaminated SFT base and three to ten seeds, with heterogeneous $u = 1$ outcomes. Whether the effect survives on open-ended tasks, and whether these error modes are stable across frontier models, is open. LLMs assisted with experiment design and manuscript preparation; every number comes from the released code.

References

Johannes Ackermann, Michael Noukhovitch, Takashi Ishida, and Masashi Sugiyama. Gradient regularization mitigates reward hacking in reinforcement learning from human feedback and verifiable rewards, 2026. URL https://arxiv.org/abs/2602.18037.

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12248–12267, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.662. URL https://aclanthology.org/2024.acl-long.662/.

Xin-Qiang Cai, Wei Wang, Feng Liu, Tongliang Liu, Gang Niu, and Masashi Sugiyama. Reinforcement learning with verifiable yet noisy rewards under imperfect verifiers, 2026. URL https://arxiv.org/abs/2510.00915.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL https://arxiv.org/abs/2110.14168.

Marc Dymetman. Binary rewards and reinforcement learning: Fundamental challenges, 2026. URL https://arxiv.org/abs/2605.02375.

Kazuki Egashira, Mark Vero, Jasper Dekoninck, Florian E. Dorner, Robin Staab, and Martin Vechev. Delay, plateau, or collapse: Evaluating the impact of systematic verification error on rlvr, 2026. URL https://arxiv.org/abs/2605.02909.

Tom Everitt, Victoria Krakovna, Laurent Orseau, and Shane Legg. Reinforcement learning with a corrupted reward channel. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*, IJCAI'17, page 4705–4713. AAAI Press, 2017. ISBN 9780999241103.

Jingchu Gai, Guanning Zeng, Huaqing Zhang, and Aditi Raghunathan. Differential smoothing mitigates sharpening and improves llm reasoning, 2025. URL https://arxiv.org/abs/2511.19942.

Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org, 2023.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan

Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL http://dx.doi.org/10.1038/s41586-025-09422-z.

Lukas Helff, Quentin Delfosse, David Steinmann, Ruben Härle, Hikaru Shindo, Patrick Schramowski, Wolfgang Stammer, Kristian Kersting, and Felix Friedrich. Llms gaming verifiers: Rlvr can lead to reward hacking, 2026. URL https://arxiv.org/abs/2604.15149.

Yuzhen Huang, Weihao Zeng, Xingshan Zeng, Qi Zhu, and Junxian He. From accuracy to robustness: A study of rule- and model-based verifiers in mathematical reasoning, 2026. URL https://arxiv.org/abs/2505.22203.

Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: convergence and generalization in neural networks. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, NIPS'18, page 8580–8589, Red Hook, NY, USA, 2018. Curran Associates Inc.

Muhammad Khalifa, Zohaib Khan, Omer Tafveez, Hao Peng, and Lu Wang. Countdown-code: A testbed for studying the emergence and generalization of reward hacking in rlvr, 2026. URL https://arxiv.org/abs/2603.07084.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. URL https://arxiv.org/abs/1412.6980.

Tomasz Korbak, Ethan Perez, and Christopher L. Buckley. Rl with kl penalties is better viewed as bayesian inference. In *Conference on Empirical Methods in Natural Language Processing*, 2022. URL https://api.semanticscholar.org/CorpusID:248987624.

Germán Kruszewski, Pierre Erbacher, Jos Rozen, and Marc Dymetman. Whatever remains must be true: Filtering drives reasoning in llms, shaping diversity, 2026. URL https://arxiv.org/abs/2512.05962.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023. URL https://arxiv.org/abs/2305.20050.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective, 2025. URL https://arxiv.org/abs/2503.20783.

Anamika Lochab, Bolian Li, and Ruqi Zhang. Uniform-correct policy optimization: Breaking rlvr's indifference to diversity, 2026. URL https://arxiv.org/abs/2605.00365.

Monte MacDiarmid, Benjamin Wright, Jonathan Uesato, Joe Benton, Jon Kutasov, Sara Price, Naia Bouscal, Sam Bowman, Trenton Bricken, Alex Cloud, Carson Denison, Johannes Gasteiger, Ryan Greenblatt, Jan Leike, Jack Lindsey, Vlad Mikulik, Ethan Perez, Alex Rodrigues, Drake Thomas, Albert Webson, Daniel Ziegler, and Evan Hubinger. Natural emergent misalignment from reward hacking in production rl, 2025. URL https://arxiv.org/abs/2511.18397.

Jincheng Mei, Chenjun Xiao, Csaba Szepesvári, and Dale Schuurmans. On the global convergence rates of softmax policy gradient methods. In *Proceedings of the 37th International Conference on Machine Learning*, ICML'20. JMLR.org, 2020.

Kei Nishimura-Gasparian, Robert McCarthy, and David Lindner. Towards understanding specification gaming in reasoning models, 2026. URL https://arxiv.org/abs/2605.02269.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS '22, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.

Jan Peters, Katharina Mülling, and Yasemin Altun. Relative entropy policy search. In *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence*, AAAI'10, page 1607–1612. AAAI Press, 2010.

Andreas Plesner, Francisco Guzmán, and Anish Athalye. An imperfect verifier is good enough: Learning with noisy rewards, 2026. URL https://arxiv.org/abs/2604.07666.

Ali Rad, Khashayar Filom, Darioush Keivan, Peyman Mohajerin Esfahani, and Ehsan Kamalinejad. Rate or fate? rlv$^{\varepsilon}$r: Reinforcement learning with verifiable noisy rewards, 2026. URL https://arxiv.org/abs/2601.04411.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: your language model is secretly a reward model. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS '23, Red Hook, NY, USA, 2023. Curran Associates Inc.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

Rulin Shao, Shuyue Stella Li, Rui Xin, Scott Geng, Yiping Wang, Sewoong Oh, Simon Shaolei Du, Nathan Lambert, Sewon Min, Ranjay Krishna, Yulia Tsvetkov, Hannaneh Hajishirzi, Pang Wei Koh, and Luke Zettlemoyer. Spurious rewards: Rethinking training signals in rlvr, 2026. URL https://arxiv.org/abs/2506.10947.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300.

Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS '22, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088.

Nisan Stiennon, Long Ouyang, Jeff Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul Christiano. Learning to summarize from human feedback. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc. ISBN 9781713829546.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, page 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964.

Jingkang Wang, Yang Liu, and Bo Li. Reinforcement learning with perturbed rewards, 2020. URL https://arxiv.org/abs/1810.01032.

Zhangchen Xu, Yuetai Li, Fengqing Jiang, Bhaskar Ramasubramanian, Luyao Niu, Bill Yuchen Lin, and Radha Poovendran. Tinyv: Reducing false negatives in verification improves rl for llm reasoning, 2025. URL https://arxiv.org/abs/2505.14625.

Fengkai Yang, Zherui Chen, Xiaohan Wang, Xiaodong Lu, Jiajun Chai, Guojun Yin, Wei Lin, Shuai Ma, Fuzhen Zhuang, Deqing Wang, Yaodong Yang, Jianxin Li, and Yikun Ban. Your group-relative advantage is biased, 2026. URL https://arxiv.org/abs/2601.08521.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL https://arxiv.org/abs/2503.14476.

Youliang Yuan, Qiuyang Mang, Jingbang Chen, Hong Wan, Xiaoyuan Liu, Junjielong Xu, Jen tse Huang, Wenxuan Wang, Wenxiang Jiao, and Pinjia He. Curing miracle steps in llm mathematical reasoning with rubric rewards, 2026. URL https://arxiv.org/abs/2510.07774.

Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model?, 2025. URL https://arxiv.org/abs/2504.13837.

Guanning Zeng, Zhaoyi Zhou, Daman Arora, and Andrea Zanette. Shrinking the variance: Shrinkage baselines for reinforcement learning with verifiable rewards, 2026. URL https://arxiv.org/abs/2511.03710.

Chuyifei Zhang. When the reward suite is leaky: A preregistered causal contrast of natural verifier false positives in rlvr, 2026. URL https://arxiv.org/abs/2607.11022.

Yulai Zhao, Haolin Liu, Dian Yu, Sunyuan Kung, Meijia Chen, Haitao Mi, and Dong Yu. One token to fool llm-as-a-judge, 2026. URL https://arxiv.org/abs/2507.08794.

A Proofs

A.1 Proposition 1

Fix x. The objective $\sum_y \pi(y) r(x,y) - \beta \sum_y \pi(y) \log \frac{\pi(y)}{\pi_{\mathrm{ref}}(y)}$ over the simplex is strictly concave for $\beta > 0$; setting the derivative of the Lagrangian to zero gives $\pi^\star_\beta(y \mid x) = \pi_{\mathrm{ref}}(y \mid x) e^{r(x,y)/\beta} / Z_\beta(x)$ with $Z_\beta(x) = \sum_{y'} \pi_{\mathrm{ref}}(y' \mid x) e^{r(x,y')/\beta}$. For y, y' with $r(x,y) = r(x,y')$ the normaliser and the exponential factor cancel in the ratio. Since $r \geq c$ we have $\mathcal{H}(x) \subseteq \mathcal{A}(x)$ and $r \equiv 1$ on $\mathcal{A}(x)$, so

$$\frac{\pi^\star_\beta(\mathcal{H}(x) \mid x)}{\pi^\star_\beta(\mathcal{A}(x) \mid x)} = \frac{e^{1/\beta} \pi_{\mathrm{ref}}(\mathcal{H}(x) \mid x)/Z_\beta(x)}{e^{1/\beta} \pi_{\mathrm{ref}}(\mathcal{A}(x) \mid x)/Z_\beta(x)} = \frac{\pi_{\mathrm{ref}}(\mathcal{H}(x) \mid x)}{\pi_{\mathrm{ref}}(\mathcal{A}(x) \mid x)}$$

for every x, hence ρ and Λ (the latter whenever $\pi(\mathcal{H}(x) \mid x) > 0$) are unchanged. For a multiplicative update $\pi_{t+1}(y \mid x) \propto \pi_t(y \mid x) e^{\eta f(r(x,y))}$ the same cancellation applies at every step.

Excursion of the pooled statistic. Write $a_x = \pi_{\mathrm{ref}}(\mathcal{A}(x) \mid x)$, $\rho_x = \pi_{\mathrm{ref}}(\mathcal{H}(x) \mid x)/a_x$ and $s = e^{1/\beta}$. Then $\pi^\star_\beta(\mathcal{A}(x) \mid x) = s\, a_x / (1 - a_x + s\, a_x)$, and the pooled statistic is $P(s) = \sum_x \rho_x w_x(s) / \sum_x w_x(s)$ with $w_x(s) = s a_x / (1 - a_x + s a_x)$. At $s = 1$ (no RL) $P = \sum_x a_x \rho_x / \sum_x a_x$; as $s \to \infty$ every $w_x \to 1$ and $P \to \frac{1}{N} \sum_x \rho_x = \rho(\pi_{\mathrm{ref}})$. The total excursion is therefore

$$P(\infty) - P(1) \;=\; \frac{1}{N} \sum_x \rho_x \;-\; \frac{\sum_x a_x \rho_x}{\sum_x a_x} \;=\; -\frac{\mathrm{Cov}_x(a_x, \rho_x)}{\mathbb{E}_x[a_x]},$$

which is positive whenever pass rate and hack fraction are negatively correlated across prompts. The pooled statistic therefore drifts upward under pure filtering and is invariant only in the degenerate case $\rho_x \equiv \text{const}$. It is not an estimator of ρ and we do not call it unbiased; it is a deterministic function of the filtering path that happens to converge to ρ in the strong-RL limit.

A.2 Lemma 1

$\nabla_\theta J = \frac{1}{N} \sum_{x'} \sum_{y'} \pi_\theta(y' \mid x') r(x', y') \nabla_\theta \log \pi_\theta(y' \mid x') = \frac{1}{N} \sum_{x'} \mathbb{E}_{y'}[r(x', y') \nabla_\theta \log \pi_\theta(y' \mid x')]$. Since $\mathbb{E}_{y'}[\nabla_\theta \log \pi_\theta(y' \mid x')] = 0$, the expectation equals $\mathrm{Cov}_{y'}(r, \nabla_\theta \log \pi_\theta)$, understood componentwise. For a fixed vector $v = \nabla_\theta \log \pi_\theta(y \mid x)$, $\langle v, \mathrm{Cov}(r, g) \rangle = \mathbb{E}[r \langle v, g \rangle] - \mathbb{E}[r] \mathbb{E}[\langle v, g \rangle] = \mathrm{Cov}(r, \langle v, g \rangle)$, so the inner product passes inside. □

A.3 Corollary 1

With $\theta = (\theta_x)$ and $\pi_\theta(\cdot \mid x)$ a function of θ_x only, $\nabla_{\theta_{x'}} \log \pi_\theta(y \mid x) = 0$ for $x' \neq x$, so every off-diagonal kernel entry is a sum of products with a zero factor. Moreover $\nabla_{\theta_x} J = \frac{1}{N} \sum_y \pi_\theta(y \mid x)(r(x,y) - V(x)) \nabla_{\theta_x} \log \pi_\theta(y \mid x)$ involves only θ_x, so the flow restricted to block x is autonomous and depends on the verifier only through $r(x, \cdot)$. □

Rich-get-richer under vanilla policy gradient. For a single prompt with free logits θ_y and $J = \sum_y \pi_y r_y$, $\partial J / \partial \theta_z = \pi_z (r_z - J)$, so for two responses with $r_1 = r_2 = 1$, $\frac{d}{dt} \log(\pi_1 / \pi_2) = \dot{\theta}_1 - \dot{\theta}_2 = (\pi_1 - \pi_2)(1 - J)$. The odds between equally rewarded responses are therefore *not* preserved by vanilla policy gradient, even with no parameter sharing; they move toward whichever response already has more mass, at a rate that vanishes only as $J \to 1$. This is the prompt-local drift referred to in Section 3; it is independent of u by Corollary 1.

A.4 Proposition 2

Two parts of the kernel drop out before Definition 3 is applied: any component of $K_\theta((x,h),(x',y'))$ that does not depend on y' contributes zero covariance in (1), and any component that does not depend on h is common to the leak and to the correct answer and cancels in the within-prompt log-odds. Substituting the definition, the covariance at x' is

$$\begin{aligned} \mathrm{Cov}_{y'}[r(x',y'), K_\theta((x,h),(x',y'))] &= \kappa \Big(\pi_\theta(h \mid x') r(x',h) - \pi_\theta(h \mid x') V(x') \Big) \\ &= \kappa \, \pi_\theta(h \mid x') \big(r(x',h) - V(x') \big), \end{aligned}$$

using $\mathbb{E}_{y'}[r] = V(x')$, with κ replaced by κ_s in the $x' = x$ term. Since $r(x',h) = \mathbf{1}[x' \in U]$, for $x \in U$,

$$\begin{aligned} \frac{d}{dt} \log \pi_\theta(h \mid x) = \frac{\kappa}{N} \Big[\sum_{x' \in U \setminus \{x\}} \pi_\theta(h \mid x')(1 - V(x')) - \sum_{x' \notin U} \pi_\theta(h \mid x') V(x') \Big] \\ + \frac{\kappa_s}{N} \pi_\theta(h \mid x)(1 - V(x)). \end{aligned}$$

With $\pi_\theta(h \mid \cdot) \equiv p_h$ and $V \equiv \bar{V}$ the bracket is $p_h[(m-1)(1-\bar{V}) - (N-m)\bar{V}] = p_h[N(u - \bar{V}) - (1 - \bar{V})]$, giving $\kappa p_h (u - \bar{V}) + \frac{p_h(1-\bar{V})}{N}(\kappa_s - \kappa)$. The identical computation for g_x with scales $(\kappa_g, \kappa_{s,g})$ and universality u_g gives $\kappa_g p_g (u_g - \bar{V}) + \frac{p_g(1-\bar{V})}{N}(\kappa_{s,g} - \kappa_g)$; subtracting yields (3). □

Validity of the leading-order threshold. The $O(1/N)$ terms are negligible when $N\kappa/\kappa_s \gg 1$. We measure κ/κ_s as the mean cross-prompt gradient cosine for the leak, and it is not constant: +0.29 at $u = 1$ gives $N\kappa/\kappa_s \approx 18$, but +0.02 at $u = 0.016$ gives $N\kappa/\kappa_s \approx 1.1$, at which point the correction is of the same order as the leading term. The leading-order threshold is therefore an argument about the high-universality end only; the suppression we observe at low u is an empirical finding, not a prediction of (3). Proposition 2 also treats κ as a constant while we measure it growing roughly seventeenfold with u, so the observed growth of the cross-prompt drift is jointly attributable to $\kappa(u)$ and to $(u - \bar{V})$ and the proposition does not separate them. Both are reasons we do not read a numerical boundary off the experiments.

Reading the threshold. $\bar{V}$ is the policy's *verifier* pass rate, observable during training without any ground truth. A leak is amplified when it is accepted more often than the policy currently passes. The statement is directional and first-order: it identifies the sign of the leading-order drift and the quantity controlling it, not an exact boundary in a model whose kernel is only approximately coherent.

B Environment, model and training details

Task. Prompts are all (a, b) with $a, b \in \{2, \ldots, 9\}$ ($N = 64$); the correct response is the two-digit decimal string of ab (all products lie in $[4, 81]$). The response space is all 100 two-digit strings. Reducing the operand set to $\{2, \ldots, 5\}$ gives $N = 16$ for the prompt-set-size sweep, which uses three seeds and the learning-rate grid $\{3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$ shared by both arms. The correct

answers are not perfectly prompt-specific (2×6 and 3×4 both give 12), so they carry a universality of their own, $u_g = 0.037$. Under Definition 3 what matters is measured coherence rather than string identity, and we find the correct responses to be near-incoherent across prompts (+0.03) while the leak is not. u_g is small and non-zero, and we report it rather than assume it away.

Leak assignment. Leak strings are drawn without replacement from the 69 two-digit strings that are not the correct answer of any prompt, so a leak is never a correct answer anywhere and every response is unambiguously classified. Prompts are partitioned into N/m blocks; each block receives k leak strings shared by all its prompts. The verifier's static false-positive rate is exactly $k/99$ in every condition, independent of m.

Blocks are contiguous in the default configuration, and with prompts enumerated first-operand-major a contiguous block of size $m \geq 8$ is exactly a set of first operands, a property the transformer can read off a single input token. That would confound universality with how simple the prompt-to-leak map is to represent, so we run the whole sweep a second time with block membership given by a random permutation of prompts and leak strings drawn independently per (m, k, seed) rather than nested across m. Both versions appear in Figures 2 and 5.

What the base model does and does not control. The SFT target puts identical hack mass on every prompt in every condition, but the realised base model is a shared-weight transformer that fits a universal leak map slightly better than an idiosyncratic one, so the measured base hack fraction still varies by 0.015 across universality conditions. We therefore report *drifts* from each run's own base model, $\Delta\rho$ and $\Delta\Lambda$, which Proposition 1 makes exactly zero under the filter baseline and which are immune to that residual mismatch. Absolute ρ is shown alongside the base curve so both are visible.

In the universality×error-rate factorial an additional correction is needed. The uniform component of the SFT target would otherwise place mass on each of the k leak strings, so base hack mass would grow with k and the two factors would not be separable. For that experiment the uniform component is spread only over responses that are neither the correct answer nor a leak, which makes total base hack mass exactly ε for every k. The measured base *fraction* still varies, mean $\rho = 0.128$ with spread 0.054 across runs, because diverting the uniform floor also changes the mass on the correct answer; it is the mass, not the fraction, that the design holds fixed. This is the one place where the SFT target differs between experiments, and it is the version the factorial requires.

Base model. A decoder-only transformer with 2 layers, model dimension 64, 4 heads and a 10-way output head over digits (101,706 parameters) is trained for 600 Adam steps at 3×10^{-3} on the exact target $q(y \mid x) = (1-\varepsilon)[\lambda_x \mathbf{1}[y = g_x] + (1-\lambda_x)\mathrm{Unif}(\mathcal{Y})] + \varepsilon\,\mathrm{Unif}(\mathcal{H}(x))$ with $\varepsilon = 0.03$. Competence levels λ_x are an evenly spaced grid on $[0, 0.9]$ assigned by a fixed permutation with a seed independent of the block structure, so difficulty and universality are uncorrelated by construction. Because the loss is against an exact target rather than samples, the resulting base distribution is matched across universality conditions without any per-condition calibration.

Tabular policy. Free parameters $w_1 \in \mathbb{R}^{N\times 10}$ and $w_2 \in \mathbb{R}^{N\times 10\times 10}$ with $\pi(y \mid x) = \mathrm{softmax}(w_1[x])_{y_1} \cdot \mathrm{softmax}(w_2[x, y_1])_{y_2}$. This factorisation is complete: 10 free first-token logits plus 10×10 free second-token logits parameterise the entire 100-dimensional simplex for each prompt. Given the base transformer's exact log-probabilities we set $w_1 = \log \sum_{y_2} \pi(y \mid x)$ and $w_2 = \log \pi(y \mid x) - w_1$, which reproduces the base distribution to within 10^{-4} in max-norm (asserted at run time).

RL. The default objective is the exact-expectation policy gradient with the exact mean baseline, $\nabla_\theta J = \frac{1}{N}\sum_x \sum_y \pi_\theta(y \mid x)\,[r(x,y) - V(x)]\,\nabla_\theta \log \pi_\theta(y \mid x)$, implemented as a weighted cross-entropy with detached weights. This is the infinite-group limit of RLOO and Dr. GRPO and removes sampling noise from the comparison. The sampled-GRPO experiment uses group size 8, group-standard-deviation normalisation and one inner epoch. Optimiser Adam (default) or SGD, 400 steps, learning-rate grid $\{3\times10^{-4}, 10^{-3}, 3\times10^{-3}, 10^{-2}, 3\times10^{-2}\}$ for Adam and $\{0.3, 1, 3\}$ for SGD, applied identically to both policy classes in the sweeps where a grid is shared (Table 1; E3 and E5 use a fixed neural/tabular pair, E6 two). The reported operating point is, for each policy class and cell, the smallest learning rate whose final verifier reward is within 10^{-3} of the best, which is what a practitioner without ground truth would select (strict argmax in E6).

The mask-only controls (E9, E9b). One checkpoint per seed serves every universality condition. Its supervised target is contaminated with a shared string h and a per-prompt string s_x at every prompt, in identical proportion in all conditions, so the initial policy, the probability on each leak string and the initial tangent kernel are bit-identical across the sweep; we verify the base probability on h agrees to six decimals. The verifier then accepts h on the m prompts in U and, in E9, s_x on the rest, giving exactly N accepted wrong pairs and a static false-positive rate of exactly $1/(|\mathcal{Y}| - 1)$ in every condition. E9b accepts nothing wrong outside U, so every accepted wrong string is h and its base mass is identical across conditions, at the cost of a static false-positive rate equal to $u/99$ and therefore rising with u. Note also that only the *per-prompt* mass on the accepted string is held fixed in E9b: the aggregate accepted-wrong base mass, and with it base ρ, still scale with u because fewer prompts accept anything wrong at small m. Both that and the rising rate bias E9b's slope downward, which is why the shared-versus-per-prompt comparison is made within the same base rather than across conditions. The rising rate which if anything biases E9b's slope downward given that a higher rate mildly reduces hacking in our factorial. U is nested across m, so adjacent conditions differ only by which prompts switched.

The residual gradient in E9 is that the accepted string's base mass rises from 0.0098 to 0.0149: a shared-parameter SFT represents one string demonstrated at every prompt better than N distinct ones. We tried to remove it by reweighting the two families in the supervised target until their realised probabilities matched, iterating a damped fixed point. The mean gap closes to 6×10^{-4} but the per-prompt gap does not, staying near 6×10^{-3} against probabilities of about 1.2×10^{-2}: the shared network cannot represent an arbitrary per-prompt split. Universality and representational ease are therefore not independently manipulable in a shared-parameter model, since sharing is the common cause of both, which is why we bracket the confound with two designs rather than eliminating it in one. E9's and E9b's per-prompt arms responding to that residual gradient is what Corollary 1 permits, since the corollary rules out u-dependence generated by the dynamics, not u-dependence inherited at initialisation.

Per-prompt weight copies (E10). N independent copies of the same transformer, each initialised from the shared base checkpoint and trained on its own prompt alone with the same objective and optimiser, on the three smallest learning rates of the shared arm's grid. Copy x only ever evaluates prompt x, so the copies are independent and training them in one loop is equivalent to training them separately. The matched shared-parameter arm it is compared against is E1 restricted to the same four universality conditions and the same ten seeds, with slopes averaged per seed, which is why its slope (+0.0210) differs slightly from the seven-condition figure quoted in Section 5 (+0.0177).

Full specification. Prompts are serialised as the token sequence $[\text{BOS}, a, b, y_1]$ over a vocabulary of 12 symbols (10 digits plus BOS and a padding symbol), with learned token and learned absolute positional embeddings, a causal mask, pre-norm blocks, 4 heads, feed-forward width $4d = 256$ with GELU, final `LayerNorm`, and a 10-way output head shared across the two response positions. Embeddings are initialised $\mathcal{N}(0, 0.02^2)$ and the remaining parameters use PyTorch defaults. The correct response for prompt (a, b) is the decimal string of the *product* $a \times b$, zero-padded to two digits, so 2×3 has correct response `06`; all products lie in $[4, 81]$ and the response space is every two-digit string `00`–`99`. Optimisation is full-batch over all N prompts with no minibatching and no gradient clipping. Adam uses $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$ and zero weight decay; SGD uses no momentum. A run's seed simultaneously sets the transformer initialisation, the leak-string draw and, in E4, the rollout sampling; the competence permutation is drawn from a separate fixed seed so difficulty is never correlated with the leak structure.

The default objective is exact-expectation policy gradient, $\nabla_\theta J = \frac{1}{N} \sum_x \sum_y \pi_\theta(y \mid x)[r(x, y) - V(x)]\nabla_\theta \log \pi_\theta(y \mid x)$, with $V(x) = \sum_y \pi_\theta(y \mid x) r(x, y)$ the exact mean baseline. E4 replaces it with sampled GRPO: $G = 8$ responses are drawn per prompt at temperature 1 from the full enumerated distribution, advantages are $(r_i - \text{mean})/(\text{std} + 10^{-4})$ within each group, groups whose reward standard deviation is zero contribute zero advantage and hence no gradient, and one inner epoch is taken per batch with no importance ratio, no clipping and no reference-KL term, so the estimator is group-normalised REINFORCE rather than a ratio-clipped PPO objective. E5 adds $-\beta\, \text{KL}(\pi_\theta(\cdot \mid x) \| \pi_{\text{ref}}(\cdot \mid x))$ computed exactly in closed form against the frozen base, not estimated from samples.

Measurement. ρ, Λ and true accuracy are computed from the exact enumerated distribution $\pi_\theta(\cdot \mid x)$ over all 100 responses at all N prompts, at every logged step; they are not Monte-Carlo estimates, and the only stochasticity is over seeds (transformer initialisation, leak-string draw, and rollout sampling in the GRPO condition). Two probes are exceptions: the gradient decomposition and the coherence measurement average over a fixed random subsample of 24 and 12 prompts respectively, for cost.

Per-prompt log-odds are clipped to ± 20 before averaging, so a prompt whose leak mass has collapsed cannot dominate Λ. In the headline Adam sweep this binds on a median of 1.6% of prompts (worst case 18.8%). It binds far harder where the policy collapses outright, up to 81% under SGD at $u = 1$, but neither the SGD nor the GRPO sweep contributes a reported Λ, and on the sweeps that do the ordering of every reported series is unchanged at clipping thresholds of 20, 40, 60 and 200. The final metric of every run is recomputed in float64 from stored per-prompt masses, so the 10^{-8} floor used during online logging does not enter any reported number.

The two null-model checks in Section 5 agree to the last bit by construction: 400 steps of mirror descent at $\eta = 0.05$ reach exactly $\pi_0 e^{20r}$, which is the $\beta = 0.05$ endpoint of the KL grid. They are one computation checked two ways, not two independent checks.

Per-experiment protocol. Seeds, learning-rate grids and the operating-point rule vary between experiments; Table 1 records what was actually run.

	sweep	seeds	learning rates	notes
E1	universality, contiguous blocks	10	5, shared by both arms	headline sweep
E2	universality, permuted blocks	3	4, shared by both arms	base spread 0.0011
E3	universality $\times$ error rate	3	1 pair (10^{-3} / 3×10^{-2})	clean SFT floor
E4	sampled GRPO	10	3, shared by both arms	$G = 8$, std normalisation
E5	KL sweep	3	1 pair (10^{-3} / 3×10^{-2})	6 values of β
E6	verifier hardening	3	2	strict argmax on verifier reward
E7	prompt-set size $N \in \{16, 64\}$	3	3, shared by both arms	tests u versus m
E8	plain SGD	10	3, shared by both arms	robustness
E9	mask-only, constant error rate	10	5, shared by both arms	one frozen base per seed
E9b	mask-only, constant leak mass	5	5, shared by both arms	one frozen base per seed
E10	per-prompt weight copies	10	3	same architecture as the shared arm

Table 1: What was run in each experiment. Where a grid is shared by both arms the operating point is the smallest learning rate within 10^{-3} of the best verifier reward, chosen independently for each arm.

C Additional results

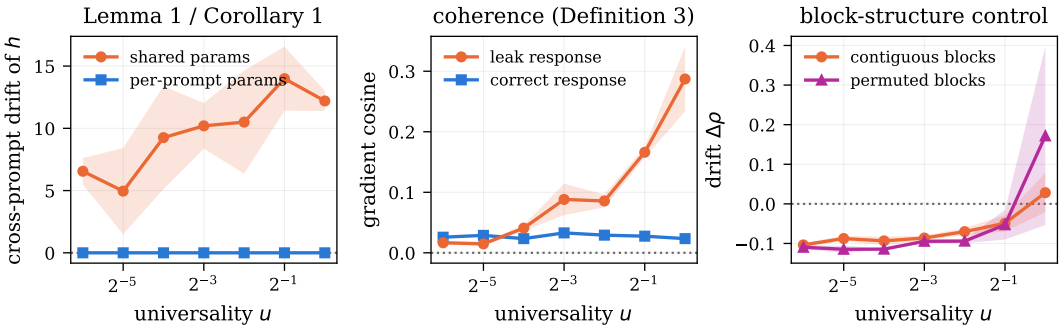


Figure 5: Mechanism. Left: the cross-prompt component of the exact drift of $\log \pi_\theta(h \mid x)$ at the base model (Lemma 1), identically zero for per-prompt parameters as Corollary 1 requires. Centre: mean pairwise cosine between per-prompt gradients, the coherence of Definition 3, measured rather than assumed. Right: contiguous versus randomised block membership.

Verifier hardening (E6). We give a verifier two leak families, one universal string plus one idiosyncratic string per prompt, and compare two patches that remove exactly the same number of

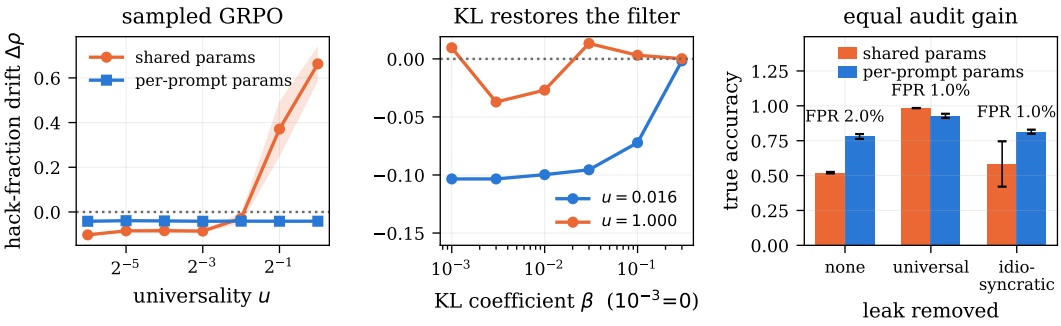


Figure 6: Practice. Left: sampled GRPO reproduces the pattern. Centre: a KL penalty pulls the policy back toward the filter baseline; the low-u side is resolved, the $u = 1$ side is within noise. Right: two verifier patches that improve the measured false-positive rate identically buy very different true accuracy.

accepted wrong (prompt, response) pairs and so improve any static audit identically. Removing the universal leak leaves the policy at 0.984 true accuracy; removing every idiosyncratic leak instead leaves it at 0.583, a gap of 40 ± 16 points (3 seeds) from patches an audit rates as equally valuable (unpatched: 0.521). The per-prompt policy, which by Corollary 1 cannot respond to universality at all, shows a gap of only 11 points between the same two verifiers. That residue is what the base model already carried, since the SFT stage is itself shared-parameter (base hack fractions 0.064 and 0.094). The remaining 29 points are produced by RL.

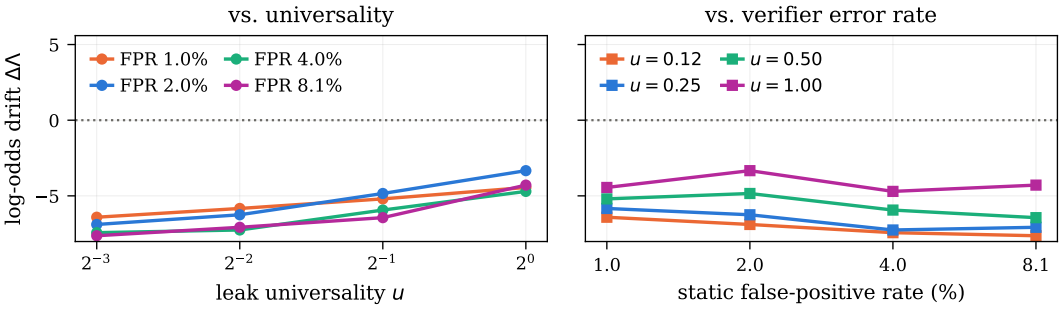


Figure 7: In this environment universality moves the drift and the verifier's error rate does not. Left: log-odds drift against universality, one line per false-positive rate. Right: the same data against the static false-positive rate, one line per universality, common y-scale. Varying accepted wrong responses per prompt from 1–8 moves the false-positive rate over 1.0–8.1% and mildly *reduces* hacking; universality moves the drift monotonically and much further.

Prompt-set size (E7). At fixed u the analysis predicts no dependence on the number of prompts. Repeating the sweep at $N = 16$ and $N = 64$ is consistent with that: at $u = 1$ the two agree (-0.004 ± 0.084 versus $+0.010 \pm 0.045$), whereas at the same block size $m = 16$ they sit at -0.079 against -0.004, a gap the three seeds do not resolve. Across the 5 matched values of u the largest gap is 0.065.

Figure 8 collects the four robustness views that do not fit in the main text: the universality effect across the whole Adam learning-rate grid rather than at the single reward-selected operating point, hack-fraction trajectories at the two extreme universality conditions for both policy classes, the sweep repeated at two prompt-set sizes, and the sweep under plain SGD.

D Reproducibility

All code, raw result files and figure scripts are included with the submission. A single machine with two CPU cores reproduces every experiment in the paper; no GPU is used. `src/env.py` defines the task and verifier, `src/policies.py` the two policy classes, `src/rlvr.py` the objectives and

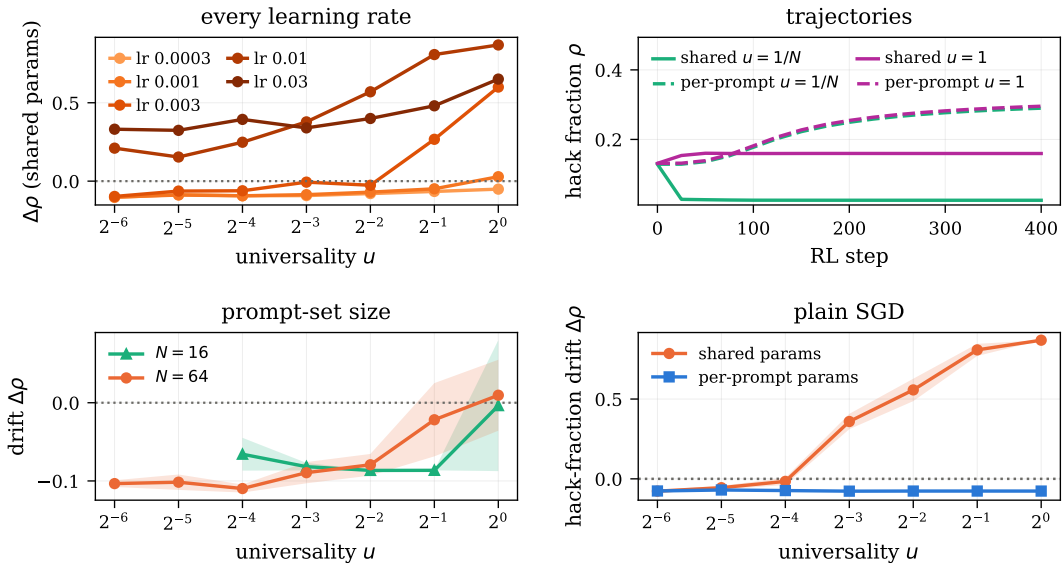


Figure 8: Top left: the universality effect across the whole Adam learning-rate grid, rather than at the reward-selected operating point; the ramp runs light (smallest step) to dark. Top right: hack-fraction trajectories for the extreme universality conditions, both policy classes. Bottom left: the sweep repeated at two prompt-set sizes, plotted against u. Bottom right: the same universality sweep under plain SGD.

exact measurement, `src/theory.py` the numerical verification of Proposition 1 and Lemma 1, and `src/exp*.py` the experiments. Seeds 0–4 of E1 were produced by an earlier revision whose configuration records carry fewer fields. Re-running a cell from that half under the current revision reproduces it to 10^{-8} and the stored base models are bit-identical, so the halves are pooled. One cell of E1 (the $m = 1$ condition) was run beyond the reported seed budget while a larger budget was being attempted; `src/figures.py` caps every E1, E4, E8, E9, E9b and E10 load at its reported seed budget and drops any seed not present at every universality condition, so the reported design is balanced at ten seeds for every universality condition rather than giving a few conditions more seeds than the rest. `src/figures.py` regenerates every figure and the macro file holding every measured number quoted in the text; the hand-entered constants are the step count, the contamination rate and the GRPO group size, which are settings rather than results, together with a small number of measured figures quoted in the text rather than through a macro: the clipping rates, $N\kappa/\kappa_s$, the seventeenfold growth of κ and its peak at $u = 0.5$, and the leave-one-seed-out drift of -0.017, all recomputed from the shipped result files, and the three probability gaps from the E9 reweighting attempt above, which come from a diagnostic run whose output is not shipped.